Lean — одно из многих средств для интерактивных доказательств. Все они устроены похожим образом: в основе лежит некий минималистичный функциональный язык программирования, программы на котором соответствуют доказательствам в некоторой конструктивной логике; набор некоторых внешних для этого языка средств упрощает создание доказательств. В доказательствах с помощью LLM чаще всего используют именно Lean по следующим причинам:

1. Много уже существующего кода для обучения нейронных сетей («большой датасет»);
2. Использование принципов классической математики, упрощающее перевод;
3. Крупнейшая библиотека формализованной математики Mathlib, на которую можно ссылаться, не принимая сомнительных аксиом.

Lean в полном объёме мы займёмся в следующий раз; здесь мы рассмотрим основания языка программирования, лежащего в его сердце, и поймём, почему программы на этом языке вообще что-то доказывают.

1 Просто типизированное лямбда-исчисление

Здесь мы рассмотрим простейшее исчисление, в котором наблюдается соответствие Карри-Говарда, согласно которому программы оказываются доказательствами: просто типизированное лямбда-исчисление (STLC, simply typed lambda calculus). Чтобы произвести это соответствие, нам нужно будет классифицировать доказательства по тому, какое утверждение они доказывают; здесь нам пригодятся *типы*:

Атом	A	$\in$	$\mathcal{A}$	
Тип	S,T	$::=$	A	атомарный тип
		$\mid$	$S \to T$	тип-стрелка

Рис. 1: Грамматика типов STLC

На Рис. 1 описана *грамматика* типов, то есть правила, по которым можно получить любой тип системы. Нотация, в которой она дана, известна как *форма Бэкуса-Наура* и означает следующее:

Определение 1 (Типы STLC)**.** Зафиксируем некоторый алфавит атомарных символов $\mathcal{A}$, элемент которого мы обычно будем обозначать A. *Множеством типов* (элементы которого мы обычно будем обозначать S, T) является наименьшее множество синтаксических деревьев, удовлетворяющее следующим свойствам:

- Для всякого $A \in \mathcal{A}$ есть *атомарный тип* A, являющийся типом;
- Если S и T — типы, то $S \to T$ — тоже тип, также называющийся «стрелка из S в T».

Обратите внимание, что к типам (и к программам, к которым мы перейдём позже) правильнее относиться именно как к синтаксическим деревьям, а не просто как к строчкам. При записи таких деревьев в тексте, конечно, нужно использовать скобки для однозначного разбора:

$$(S \to T) \to U \stackrel{?}{=} S \to T \to U \stackrel{?}{=} S \to (T \to U)$$

Правда, для облегчения письма и чтения в таких случаях принимают некоторые соглашения о нотации. Так, в случае стрелки, $(\to)$ считается правоассоциативным оператором, т.е. $S \to T \to U$ понимается как $S \to (T \to U)$.

Множество програм, классифицируемое такими типами, строится как на Рис. 2.

Переменная	x	$\in$	$\mathcal{V}$	
Терм	s,t	$::=$	x	переменная как терм
		$\mid$	$\lambda x.t$	анонимная функция
		$\mid$	$t\,s$	применение функции

Рис. 2: Грамматика термов STLC

Применение функции левоассоциативно: $(s\,t)\,u$ понимается как $s\,t\,u$. Кроме этого, при последовательном объявлении анонимных функций все символы λ кроме первого можно опустить: $\lambda x\,y\,z.\,t$ понимается как $\lambda x.(\lambda y.(\lambda z.t))$.

Наконец, свяжем вместе термы и типы: скажем, какие термы t к какому типу T относятся; этим занимается *суждение о типизации*, обозначаемое $t : T$.

Идея этого суждения состоит в том, что типы — это что-то вроде множеств, а термы типа T — элементы соответствующего множества; тип-стрелка из A в B тогда не что иное, как множество функций из A в B, а $\lambda x.t$ — форма записи функции, по иксу из A возвращающей t из B. Но тогда возникает вопрос — какой тип имеет переменная x? В суждении $(\lambda x.x) : T \to T$ она, очевидно, имеет тип T; но если мы заменим T на U, то икс уже будет иметь тип U! По сути, тип икса зависит от *контекста*: списка, хранящего информацию о типе каждой переменной, используемой в текущем выражении.

Контекст	Γ	$::=$	$\cdot$	пустой контекст
		$\mid$	$\Gamma, x : T$	расширение контекста

Рис. 3: Контекст типизации STLC

Итого, суждение о типизации имеет вид $\Gamma \vdash t : T$ и читается как «в контексте Γ, терм t имеет тип T». Задаётся оно с помощью *правил вывода*, приведённых на Рис. 4.

Опять же, правила вывода можно понимать просто как очень краткую нотацию для следующего определения, по аналогии с нотацией Бэкуса-Наура:

Определение 2 (Отношение типизации в STLC)**.** Отношение типизации $\Gamma \vdash t : T$ — трёхместное отношение, связывающее вместе контекст Γ, терм t и тип T, являющееся наименьшим отношением, удовлетворяющим следующим правилам:

$$\textsc{Var}\ \frac{}{\Gamma \vdash x : T}(x : T) \in \Gamma \qquad \textsc{Abs}\ \frac{\Gamma, x : S \vdash t : T}{\Gamma \vdash \lambda x.t : S \to T} \qquad \textsc{App}\ \frac{\Gamma \vdash t : S \to T \qquad \Gamma \vdash s : S}{\Gamma \vdash t\, s : T}$$

Рис. 4: Правила типизации STLC

- Если в контексте Γ есть запись вида $x : T$, то $\Gamma \vdash x : T$.
- Если в расширенном контексте $\Gamma, x : S$ терм t имеет тип T, то $\Gamma \vdash \lambda x.t : S \to T$.
- Если в контексте Γ верно, что $t : S \to T$ и $s : S$, то $\Gamma \vdash t\, s : T$.

В целом, уже построенную систему можно рассматривать как способ формальной записи некоторых теоретико-множественных функций. Например, мы можем протипизировать композицию функций (ниже $\Gamma \triangleq f : B \to C, g : A \to B, x : A$ для краткости):

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash f : B \to C}\,\textsc{Var} \quad \dfrac{\dfrac{}{\Gamma \vdash g : A \to B}\,\textsc{Var} \quad \dfrac{}{\Gamma \vdash x : A}\,\textsc{Var}}{\Gamma \vdash g\, x : B}\,\textsc{App}}{f : B \to C, g : A \to B, x : A \vdash f(g\, x) : C}\,\textsc{App}}{f : B \to C, g : A \to B \vdash \lambda x.f(g\, x) : A \to C}\,\textsc{Abs}}{f : B \to C \vdash \lambda g\, x.f(g\, x) : (A \to B) \to A \to C}\,\textsc{Abs}}{\cdot \vdash \lambda f\, g\, x.f(g\, x) : (B \to C) \to (A \to B) \to A \to C}\,\textsc{Abs}$$

Рис. 5: Вывод типизации композиции функций

Теперь давайте сотрём все термы из вывода выше. Получим следующее дерево вывода в *импликативном фрагменте интуиционистской пропозициональной логики*:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash B \to C} \quad \dfrac{\dfrac{}{\Gamma \vdash A \to B} \quad \dfrac{}{\Gamma \vdash A}}{\Gamma \vdash B}}{B \to C, A \to B, A \vdash C}}{B \to C, A \to B \vdash A \to C}}{B \to C \vdash (A \to B) \to A \to C}}{\cdot \vdash (B \to C) \to (A \to B) \to A \to C}$$

Рис. 6: Вывод в Int

Это конкретный пример применения *соответствия Карри-Говарда*: любое суждение о типизации $\Gamma \vdash t : T$ соответствует доказательству $(\downarrow \Gamma) \vdash T$ [1]; и любое доказательство $\Gamma_0 \vdash T$

[1] Где $\downarrow \cdot = \cdot$ и $\downarrow (\Gamma, x : T) = (\downarrow \Gamma), T$.

соответствует суждению о типизации некоторого терма $(\uparrow \Gamma_0) \vdash t : T$ [2].

В чём плюс от введения нового исчисления? Во-первых, у нас теперь есть единый формализм, в рамках которого можно u строить теоретико-множественные объекты (где $A \to B$ интерпретируется как множество функций), u доказывать какие-то утверждения (где $A \to B$ интерпретируется как импликация). Во-вторых, задача проверки типизации в STLC решается алгоритмически — другими словами, по Γ, t и T можно проверить автоматически, верно ли, что $\Gamma \vdash t : T$. Так что пользователю достаточно выписать тип и программу, а корректность логического вывода среда интерактивных доказательств проверит сама.

2 Вновь о Lean

Конечно, в Lean используется чуть более сложная система, чем STLC: для содержательных математических рассуждений нам не хватает кванторов, других логических связок кроме импликации, способов строить бесконечные множества, фактормножества и так далее. Но, надеюсь, из изложения выше понятен общий принцип работы: пользователь вместе с интерактивной средой доказательств любым доступным ему образом строит некоторые t и T; ядро среды проверяет, что $t : T$. По теоретико-множественной семантике $[\![_]\!]$, доказанной[3] для Lean, полученной программе соответствует некоторый теоретико-множественный объект $[\![t]\!] \in [\![T]\!]$. Если при этом T было типом, соответствующим утверждению, то $[\![t]\!]$ оказывается доказательством этого утверждения.

Для практики с ядром Lean я составил несколько упражнений, доступных в файле `AiLeanMM/Basic.lean` в репозитории, в который я буду выкладывать все свои материалы по данному мини-курсу: `https://github.com/TurtlePU/AiLeanMM`. Для удобства работы советую настроить Lean в каком-нибудь редакторе кода; проще всего это сделать для VS Code:

1. Установите `VS Code`.
2. Установите расширение `Lean 4` для VS Code.
3. Склонируйте репозиторий `AiLeanMM` через стартовое меню VS Code.
4. Следуйте указаниям расширения Lean 4 (возможно, нужно сделать что-то с Вашим подключением к интернету, чтобы расширение смогло установить все необходимые пакеты на Ваш компьютер).

[2] Где $\uparrow \cdot = \cdot$ и $\uparrow (\Gamma, T) = (\uparrow \Gamma), x_T : T$ с точностью до выбора новых имён для переменных одинакового типа.

[3] Mario Carneiro (2019). "The Type Theory of Lean". `https://github.com/digama0/lean-type-theory/releases/tag/v1.0`. Дис. ... маг. Carnegie Mellon University